OrangeHRM UI Automation - TC001 & TC002 (code walkthrough)

Playwright + BDD + POM | aloc999/orangehrm-ui-automation-playwright | read top to bottom, ~10 minutes

Hi everyone. In this video I am going to walk you through two test cases I automated on the OrangeHRM demo site, using Playwright with BDD and the Page Object Model, and I will go through the actual code line by line so you can see exactly how each Gherkin step maps to real browser actions. Everything I show is already pushed to the main branch of orangehrm-ui-automation-playwright.

The two cases are TC001, uploading a new profile picture, a Functional test in the Profile feature with Medium priority, and TC002, searching for a non-existent user account, a Negative test in the Search feature with Low priority. Both are marked Yes for automation candidacy because they are stable, they use clear data, and their assertions are deterministic. The framework has three layers: feature files under features hold Gherkin in business language with no CSS, the step file workspace.steps.ts only orchestrates page objects, and everything about selectors and waits lives in src/pages. So if the UI changes, I only touch the Page Object.

File 1 - features/my-info.feature (this is TC001)

```
@my-info @profile
Feature: My Info profile picture
  As a logged-in user
  I need to update my profile picture from the My Info section
  So that my employee profile stays current

  Scenario: TC001 - Uploading a new profile picture shows a success message
    Given an administrator is on the My Info workspace
    When the profile picture is updated with a valid image
    # TC doc says "Successfully Saved"; app OS 5.9 actually returns "Successfully Updated".
    Then a success message "Successfully Updated" is displayed
```

This is the whole TC001 spec. The first line tags it at-my-info and at-profile so I can run just this scenario with grep. Then the Feature block states the role and the goal in plain business language. The Scenario has three steps and deliberately contains zero technical detail: no button names, no CSS, no file paths. The Given puts us on the My Info workspace. The When performs the whole upload flow with a valid image. The Then asserts the success message, and it takes the expected text as a quoted parameter, which the step definition turns into a case-insensitive regex. The comment line documents an honest discrepancy I found: the test-case document says Successfully Saved, but the demo app version 5.9 actually returns Successfully Updated from the picture-upload API, which I verified in the network log with HTTP 200, so the feature uses the real application text.

File 2 - src/pages/MyInfoPage.ts (all browser mechanics live here)

```
import { expect } from '@playwright/test';
import { BasePage } from './BasePage';

export class MyInfoPage extends BasePage {
  async open(): Promise<void> {
    const item = this.page.locator('a.oxd-main-menu-item')
      .filter({ hasText: /^My Info$/ });
    await expect(item).toBeVisible();
    await item.click({ noWaitAfter: true });
    await this.waitUntilReady();
```

```
    // My Info lives under the PIM module, so the breadcrumb reads "PIM".
    await this.expectModule(/pim/i);
    await expect(this.page).toHaveURL(/pim\/view(MyDetails|PersonalDetails)/);
  }
```

The class extends BasePage, so it inherits the shared waits. The open method starts by locating the side-menu anchor a.oxd-main-menu-item filtered by the exact text My Info with start and end anchors, so it cannot accidentally match a submenu. It asserts the item is visible, clicks it with noWaitAfter true so Playwright does not hang waiting for navigation, then calls waitUntilReady, which waits for domcontentloaded plus the OrangeHRM loading spinners to disappear. Then comes the lesson from my first failed run: I originally asserted the breadcrumb says My Info, but the breadcrumb on this page actually reads PIM, because My Info is part of the PIM module. So the code asserts expectModule for PIM and additionally checks the URL matches pim slash viewMyDetails or PersonalDetails. Two independent signals, so a wrong page cannot slip through.

```
  async openProfilePictureDialog(): Promise<void> {
    const picture = this.page.locator('.orangehrm-edit-employee-image').first();
    await expect(picture).toBeVisible();
    await picture.click();
    await this.waitUntilReady();
    await expect(this.page.getByRole('heading',
      { name: /change profile picture/i })).toBeVisible();
  }
```

The next method opens the upload form. It locates .orangehrm-edit-employee-image and takes the first match, asserts it is visible, and clicks it. Clicking navigates to the photograph page at pim slash viewPhotograph slash empNumber slash 7. After the loader settles, it asserts the heading Change Profile Picture is visible, which proves the form really rendered before we touch the file input. I captured this selector with playwright codegen by clicking the photo, but the surrounding waits and heading check are hand-written.

```
  async uploadProfilePicture(absolutePath: string): Promise<void> {
    const fileInput = this.page.locator('input.oxd-file-input');
    await expect(fileInput).toBeAttached();
    await fileInput.setInputFiles(absolutePath);
    // Preview renders as data URL -- no loader to wait for here.
    await expect(this.page.locator(
      '.orangehrm-employee-picture img').first()).toBeVisible();
  }
```

This is the upload itself. It locates the hidden file input input.oxd-file-input and only requires it to be attached to the DOM, not visible, because file inputs on this app are hidden by design and setInputFiles works on attached inputs. Then setInputFiles injects the file directly, which is the correct Playwright pattern: you must never click the Choose File button, because that opens a native OS dialog Playwright cannot control. After upload, the app renders the preview image as a base64 data URL, so the code asserts the preview image inside .orangehrm-employee-picture is visible. That assertion doubles as proof the file was accepted before we press Save. The file passed in is test-assets slash profile.png, a real 200 by 200 PNG of 589 bytes, satisfying the app hint of jpg, png or gif up to 1 megabyte.

```
  async saveProfilePicture(): Promise<void> {
    const save = this.page.getByRole('button', { name: 'Save' });
    await expect(save).toBeEnabled();
```

```
    // Toast auto-dismisses in ~5s, so don't block on loaders before asserting it.
    await save.click({ noWaitAfter: true });
  }

  async expectSuccessToast(
    expected: string | RegExp = /successfully updated/i): Promise<void> {
    const toast = this.page.locator('.oxd-toast--success, .oxd-toast');
    await expect(toast.first()).toBeVisible({ timeout: 10_000 });
    await expect(toast.first()).toContainText(/successfully/i);
    await expect(toast.first()).toContainText(expected);
  }
```

Saving and asserting are split into two methods on purpose. saveProfilePicture locates the Save button by accessible role, asserts it is enabled so we never click a dead button, and clicks with noWaitAfter true. That flag is the fix for my second failure: the success toast auto-dismisses after about five seconds, and my first version called waitUntilReady after the click, so the toast was already gone when the assertion ran and the test timed out waiting for a toast that would never come back. By clicking without waiting, control returns instantly and the toast check starts while the toast is still on screen. expectSuccessToast then locates .oxd-toast--success with a fallback to .oxd-toast, waits up to ten seconds for it to be visible, and asserts it contains the word Successfully plus the exact expected text passed from the feature file. The default parameter is Successfully Updated, matching the real API response.

File 3 - features/steps/workspace.steps.ts (TC001 glue: Given / When / Then)

```
import { resolve } from 'node:path';

Given('an administrator is on the My Info workspace',
  async ({ myInfoPage, loginPage, page }) => {
    await ensureSignedIn(page, loginPage);
    await myInfoPage.open();
});

When('the profile picture is updated with a valid image',
  async ({ myInfoPage }) => {
    await myInfoPage.openProfilePictureDialog();
    await myInfoPage.uploadProfilePicture(resolve('test-assets/profile.png'));
    await myInfoPage.saveProfilePicture();
});

Then('a success message {string} is displayed',
  async ({ myInfoPage }, expected: string) => {
    await myInfoPage.expectSuccessToast(new RegExp(expected, 'i'));
});
```

This is the glue between Gherkin and the Page Object, and notice there is not a single CSS selector or click in this file. The Given destructures the myInfoPage, loginPage and page fixtures, calls the shared ensureSignedIn helper, which navigates to the dashboard from about blank or signs in as administrator if the login page is showing, reusing the stored session so the suite does not log in for every scenario, and then delegates to myInfoPage.open. The When chains the three page methods in order: open the dialog, upload via resolve of test-assets profile.png into an absolute path, because setInputFiles requires an absolute path, then save. The Then uses the

playwright-bdd placeholder string in braces, which captures the quoted text from the feature file and converts it to a case-insensitive regex before passing it to expectSuccessToast. That is how one step definition serves any expected message.

File 4 - features/steps/fixtures.ts (where myInfoPage comes from)

```
import { MyInfoPage } from '../../src/pages/MyInfoPage';
// inside Pages type:
myInfoPage: MyInfoPage;
// inside test.extend:
myInfoPage: async ({ page }, use) => {
  await use(new MyInfoPage(page));
},
```

The myInfoPage object used above is constructed here. The fixtures file extends the base Playwright test with one factory per page object, each receiving the same page instance. So when the step asks for myInfoPage, playwright-bdd instantiates MyInfoPage bound to the current browser tab. Adding a new page to the framework is always these three moves: create the Page class, register it in fixtures, and write steps that call it.

File 5 - features/admin-users.feature + src/pages/AdminPage.ts (this is TC002)

```
Scenario: Search for an unregistered username shows the empty state
  Given an administrator is on the Admin workspace
  When system users are filtered by an unregistered username
  Then no matching user records are displayed
```

```
async filterUsersByUsername(username: string): Promise<void> {
  const usernameField = this.page.locator('.oxd-input-group')
    .filter({ hasText: 'Username' }).locator('input');
  await usernameField.fill(username);
  await this.page.getByRole('button', { name: 'Search' }).click();
  await this.waitUntilReady();
}

async expectNoUsersListed(): Promise<void> {
  // The table empty-state is a span; the toast uses a <p> -- scope to the span.
  await expect(
    this.page.locator('span.oxd-text--span')
      .filter({ hasText: 'No Records Found' }).first(),
  ).toBeVisible();
}
```

TC002 already existed in the repo and it is a negative test, so the interesting part is the empty-state assertion. The When step calls filterUsersByUsername with the literal value NonExistentUserXYZ999, exactly as the manual test case demands. Inside the method, the username field is found by taking every .oxd-input-group, filtering to the one whose text contains Username, and then locating its inner input. That is more robust than an index-based selector because the Admin filter row has several inputs. It fills the value, clicks the Search button by role, and waits for the table loader to settle. The assertion then looks for a span with class oxd-text--span containing No Records Found and takes the first match. Scoping to the span tag is critical, and the code comment says why: the success toast elsewhere uses a p tag with similar text, so an unscoped locator could false-match a leftover toast instead of the table empty state. The step Then no matching user records are displayed simply delegates to this method.

File 6 - src/pages/BasePage.ts (shared waits both pages inherit)

```
protected get loaders(): Locator {
  return this.page.locator(
    '.oxd-loading-spinner, .oxd-form-loader, .oxd-circle-loader, .oxd-table-loader',
  );
}

async waitUntilReady(): Promise<void> {
  await this.page.waitForLoadState('domcontentloaded');
  await this.loaders.first()
    .waitFor({ state: 'hidden', timeout: 20_000 })
    .catch(() => undefined);
}
```

Both page objects inherit this. The loaders getter groups all four OrangeHRM spinner classes into one locator. waitUntilReady first waits for the domcontentloaded load state, then waits for the first visible loader to become hidden within twenty seconds, swallowing the timeout if no loader ever appeared. Every navigation and every table refresh goes through this, which is why the framework has zero hard-coded sleeps. The only place I deliberately skip it is between Save and the toast check in TC001, because there the waiting itself would destroy the evidence.

File 7 - playwright.config.ts + src/setup/auth.setup.ts + src/config/env.ts (why login happens once)

```
const guest = defineBddProject({ name: 'guest',
  features: 'features/**/*.feature',
  steps: 'features/steps/**/*.ts', tags: '@guest' });
const app = defineBddProject({ name: 'app',
  features: 'features/**/*.feature',
  steps: 'features/steps/**/*.ts', tags: 'not @guest' });

export default defineConfig({
  fullyParallel: true,
  retries: process.env.CI ? 2 : 1,   // shared demo site is flaky
  timeout: 90_000, expect: { timeout: 20_000 },
  use: { baseURL: appConfig.baseUrl, trace: 'on-first-retry',
    screenshot: 'only-on-failure', video: 'retain-on-failure',
    actionTimeout: 20_000, navigationTimeout: 45_000,
    viewport: { width: 1440, height: 900 } },
  projects: [
    { name: 'setup', testDir: './src/setup', testMatch: /.*\.setup\.ts/ },
    { ...guest },
    { ...app, dependencies: ['setup'],
      use: { storageState: '.auth/admin.json' } },
  ],
});
```

This config is why every run prints 2 passed instead of 1. There are three projects. The setup project runs files under src slash setup once and logs in as administrator. The guest project runs only scenarios tagged at-guest starting logged out, for example the login page itself. The app project runs everything not tagged at-guest, declares a dependency on setup so login always finishes first, and reuses the saved session file .auth slash admin.json through storageState, so no scenario logs in again. retries is 1 locally and 2 on CI because the demo is a public

shared instance where a step can randomly take longer on the first attempt. Timeouts are explicit: 90 seconds per test, 20 seconds per assertion and per action, 45 seconds for navigation. Trace is captured on first retry, screenshot and video only on failure, all at a fixed 1440 by 900 viewport so screenshots are comparable.

```
setup('authenticate as administrator', async ({ page }) => {
  mkdirSync('.auth', { recursive: true });
  const loginPage = new LoginPage(page);
  const dashboardPage = new DashboardPage(page);
  await loginPage.open();
  await loginPage.signInAsAdministrator();
  await dashboardPage.expectLoaded();
  await page.context().storageState({ path: '.auth/admin.json' });
});
```

This is the setup file that produces the first passed line. It creates the .auth folder, opens the login page, signs in with the administrator credentials, waits until the dashboard module and URL prove login succeeded, and then dumps the browser storage state to .auth slash admin.json. Every app-project scenario then starts already authenticated. The credentials and base URL come from src slash config slash env.ts, which reads dotenv and defaults to base URL opensource-demo dot orangehrmlive dot com, username Admin, password admin123, exactly the credentials printed on the demo login page, so cp dot env dot example dot env is optional.

```
async function ensureSignedIn(page: Page, loginPage: LoginPage): Promise<void> {
  if (page.url() === 'about:blank') {
    await loginPage.openPath('/web/index.php/dashboard/index');
  }
  if (/auth\/login/.test(page.url())) {
    await loginPage.signInAsAdministrator();
  }
}
```

For completeness, this is the ensureSignedIn helper at the top of workspace.steps.ts that every Given calls. If the tab is still on about blank, it navigates to the dashboard, and if the app bounced us to auth slash login because the stored session expired, it signs in again as administrator. That is the safety net underneath the Given steps, so a scenario never starts on the wrong page.

File 8 - test-assets/profile.png + package.json scripts (the data and the commands)

```
test-assets/profile.png
  PNG image data, 200 x 200, 8-bit/color RGB, non-interlaced
  589 bytes (limit on the app: jpg/png/gif up to 1 MB)
  Recommended dimensions on the page: 200px X 200px
```

The test data is a single real image file committed to the repo, not generated at runtime. It is a 200 by 200 pixel PNG, 589 bytes, which exactly matches the recommended dimensions printed under the upload form and sits far below the 1 megabyte limit. Because it is tiny, upload over the shared demo network takes under a second, and because it is committed, CI uses byte-for-byte the same file I used locally. The step resolves it with node path resolve so setInputFiles always receives an absolute path regardless of working directory.

```
"test": "bddgen && playwright test",
"test:headed": "bddgen && playwright test --headed",
"test:smoke": "bddgen && playwright test --grep @smoke",
"typecheck": "tsc --noEmit",
```

```
"report": "playwright show-report",
"codegen": "playwright codegen https://opensource-demo.orangehrmlive.com"
```

These are the npm scripts. test runs bddgen first so the runnable specs are never stale, then the full Playwright run. test headed is the same run with a visible browser. typecheck is tsc dash dash noEmit and must be clean before any run. report opens the HTML report folder. codegen launches the recorder I used to capture the selectors for both pages. The only browser needed is Chromium, installed by npx playwright install chromium.

Running it live - one command at a time, in this exact order

Run these strictly one by one: type one command, press Enter, wait until it fully finishes and the terminal prompt returns, read the result, and only then type the next one. Never paste them all at once. Each step below tells you exactly what finished looks like.

Step 1, check Node. Type node dash dash version and press Enter. You must see version 18 or higher, ideally 20. If the command is not found, stop here and install Node first, because nothing below will work without it.

```
node --version   # expect: v18+ (v20 LTS ideal)
```

Step 2, install dependencies. Type npm install and wait several minutes on the first run until it prints added N packages. This downloads Playwright, playwright-bdd, TypeScript and dotenv into node_modules.

```
npm install
```

Step 3, install the browser. Type npx playwright install chromium and wait until the download of about 150 megabytes finishes. The tests drive this Chromium, not your personal Chrome, so the run is identical on every machine.

```
npx playwright install chromium
```

Step 4, optional env file. Copy dot env dot example to dot env on Linux or Mac, or use copy on Windows PowerShell. You may skip this entirely because the defaults already equal Admin, admin123 and the public demo URL.

```
cp .env.example .env   # Windows PowerShell: copy .env.example .env
```

Step 5, typecheck. Type npm run typecheck and press Enter. Finished looks like silence: only npm notice lines and zero red errors, because it runs tsc dash dash noEmit. If red errors appear, fix the TypeScript first and re-run this step. Do not continue while it is red.

```
npm run typecheck
```

Step 6, generate the runnable specs. Type npx bddgen and wait for the injected env line. This compiles every dot feature file plus the step files into the dot features-gen folder, and you must repeat it after any feature edit, otherwise Playwright executes the stale spec from the previous generation.

```
npx bddgen
```

Step 7, run TC001 alone. watch as I type the grep command for TC001 with one worker, press Enter, and wait about forty seconds without touching the terminal. And there it is: two green checkmarks and the line 2 passed. The first line is the login setup, the second is the profile-picture scenario. On this run the setup took about sixteen seconds and the scenario about seventeen and a half, thirty-six seconds total.

```
npx playwright test --grep "TC001" --workers=1
# this run:
```

```
#   [setup] authenticate as administrator .... 15.8s PASS
#   [app] TC001 profile picture ............... 17.5s PASS
#   2 passed (36.0s)
```

Step 8, run TC002 alone, only after step 7 has fully finished. watch again: the same pattern with the other grep and about forty seconds of waiting. And it is green too, a clean 2 passed, setup eighteen and a half seconds and scenario nineteen. Now, on a slow minute you might instead see 1 flaky plus 1 passed, which also counts as green, and I have seen that myself: the setup login failed once because the shared demo was slow, then passed on retry, while the scenario itself passed. If the word flaky ever appears while recording, just explain it calmly: flaky means failed then passed on retry, it is not a failure, and this is exactly why the config sets one retry locally and two on CI.

```
npx playwright test --grep "unregistered username" --workers=1
# expect: 2 passed  OR  1 flaky, 1 passed (both = green)
```

Step 9, optional full suite. npm test simply means bddgen followed by the whole Playwright run, so use it when you want everything, not during the one-by-one demo. Step 10, open the report with npm run report, which serves the HTML page I dissected in File 9, and open any trace zip with show-trace followed by its path.

```
npm test         # full suite = bddgen && playwright test
npm run report   # opens playwright-report/ in the browser
```

To wrap up: TC001 covers the functional happy path of uploading a picture with toast validation, and its three trickiest lines are the PIM breadcrumb assertion, the noWaitAfter click before the toast, and the span-scoped empty-state check inherited by its sibling TC002, which covers the negative search path. Both deserve automation because they are repeatable and depend on no hand-crafted data, and the suite now has ten flows, above the required seven. Thanks for watching.

File 9 - Reading playwright-report (what each row and attachment means)

```
npm run report   # serves playwright-report/index.html in the browser
# or: npx playwright show-report

SUITE  src/setup/auth.setup.ts
  SPEC  authenticate as administrator
    retry 0 .... FAILED  4.3s   [screenshot .png] [video .webm] [error-context .md]
    retry 1 .... PASSED  16.4s  [trace .zip]
SUITE  .features-gen/app/features/admin-users.feature.spec.js
  SUITE  System user administration          <- Feature title becomes a suite
    SPEC  Search for an unregistered username ...  [tag: @admin]
      retry 0 .... PASSED  15.3s   3 steps, no attachments
```

This is my real report from the flaky TC002 run, and it is the best slide in the video because it proves the retry story with evidence. Open it with npm run report, which serves the index.html inside playwright-report. On the left is the suite tree. Notice the generated spec path dot features-gen slash app slash features slash admin-users dot feature dot spec dot js: that file does not exist in git, it is compiled by bddgen from the feature plus the steps, which is why bddgen must run before every test. Under it, the Feature title System user administration becomes a nested suite, and the Scenario title becomes the spec row, carrying its at-admin tag, so tags and grep stay traceable from report back to source.

Click the setup spec and you see two result rows, one per retry. Retry 0 failed after four seconds and carries three

attachments: a PNG screenshot of the page at failure, a webm video of the whole attempt, and error-context dot md, which contains the Playwright error, the exact source line, and a full accessibility-tree snapshot of the page with every role, name and ref, exactly like the YAML I showed when debugging. Retry 1 passed in sixteen seconds and carries one attachment: a trace zip. Opening it with show-trace launches the Trace Viewer with a filmstrip timeline, DOM snapshots per action, network including the login calls, console output and the action list, so you step through what the retry did differently instead of guessing.

Then click the scenario spec: one row, passed on retry 0 in fifteen seconds, three steps visible, one per Gherkin line, Given, When, Then, and zero attachments. That absence is the point to state on camera: passing tests produce no screenshots, no video and no trace, because the config sets screenshot only-on-failure, video retain-on-failure and trace on-first-retry, keeping CI artifacts small. A green row with three steps and no attachments is the definition of a clean pass, while the setup spec above it shows failed-then-passed, which the header summarizes as 1 flaky. End the demo by typing the scenario name in the report search box to show filtering, and remind viewers the JSON behind this page is playwright-report slash results dot json, so the same statuses can be consumed by any CI dashboard.

File 9b - Narrating an all-green report on camera (TC001 + TC002)

Here is exactly what to do and say when both are green. Open the report with npm run report and start wide: the header shows 2 passed, zero failed, and the total time, about thirty-six seconds on my TC001 run and forty on TC002. Say that number aloud, because it anchors the demo. Then move to the suite tree on the left: row one is the setup login, row two is the scenario. Click the setup row first: one result, passed, about sixteen seconds, no attachments, which proves login happened once and cleanly. Then click the TC001 scenario row and read its three steps top to bottom, Given the administrator is on My Info, When the picture is updated, Then the success message is displayed, each with a green check and a sane duration, and zero attachments underneath. State the rule on camera: passing tests produce no screenshots, no video and no trace, because the config only creates them on failure, so green plus three steps plus zero attachments is the definition of a clean pass.

Do the identical clicks for TC002 and read its Given, When and Then the same way: filter by the unregistered username, assert No Records Found, green, no attachments. Then type the scenario name into the report search box to show filtering live. Finish with the verification checklist spoken aloud while pointing: one, terminal exit code zero, checked by typing echo dollar-question-mark directly with no pipe, because a pipe would report the pager exit code instead of Playwright; two, the header shows unexpected zero and flaky zero; three, every spec shows three steps, never zero, because a zero-step pass proves nothing; four, durations are teens of seconds, never a fraction of a second, because an instant pass means the assertions never ran; five, the strongest proof, change one expected string to a wrong value, rerun, watch it go red, then change it back, which proves the test can actually fail.